Séance 2

Syntaxe de base

Objectifs de la séance

- Connaître les types de base et les particularités du typage Python
- Afficher et saisir des informations, formater avec les f-strings
- Manipuler les séquences (str, list, tuple, range), les dictionnaires et les ensembles
- Comprendre l'affectation par référence, la mutabilité et les copies
- Décomposer une séquence (unpacking)

⚡ Quiz flash : séance 1

1. Quelle commande crée un environnement virtuel nommé `.venv` ?
2. Que vaut `7 // 2` ? Et `7 % 2` ?
3. Pourquoi `2nd_server = "srv-02"` provoque-t-il une erreur ?
4. Qu'affiche `print("RAM", 32, "Go")` ?
5. Quelle version de Python ne faut-il plus utiliser, et depuis quand ?

Plan de la séance

Durée	Bloc
10 min	Quiz flash
20 min	Théorie 1 : types, typage, conversions, console, f-strings
30 min	🧪 Lab partie 1
5 min	Pause
20 min	Théorie 2 : séquences, slicing, dict, set, références, copies, unpacking
30 min	🧪 Lab partie 2 + synthèse + GestParc étape 1
5 min	Récapitulatif

Les types de base

Type	Exemple	Usage
`int`	`cpu_count = 8`	Entiers, sans limite de taille
`float`	`cpu_load = 0.75`	Nombres décimaux
`str`	`hostname = "srv-web-01"`	Texte (Unicode)
`bool`	`is_online = True`	Vrai / faux
`NoneType`	`assigned_to = None`	Absence de valeur

```
>>> 2 ** 100
1267650600228229401496703205376
>>> 0.1 + 0.2
0.30000000000000004
```

Un typage dynamique… et fort

Dynamique : le type est porté par la valeur, pas par la variable.

```
status = "online"   # status désigne un str
status = 1          # puis un int : autorisé
```

Fort : pas de conversion implicite entre types incompatibles.

```
>>> "Port : " + 8080
TypeError: can only concatenate str (not "int") to str
>>> "Port : " + str(8080)
'Port : 8080'
```

Connaître le type : `type()` et `isinstance()`

```
>>> ram_gb = 32
>>> type(ram_gb)
<class 'int'>
>>> isinstance(ram_gb, int)
True
>>> isinstance(ram_gb, (int, float))   # l'un ou l'autre
True
>>> type(True)
<class 'bool'>
```

💡 On préfère `isinstance()` pour tester un type (il tiendra compte de l'héritage, vu en S9).

Introduction aux type hints

Des **annotations** qui documentent le type attendu.

```
hostname: str = "srv-web-01"
port: int = 443
cpu_load: float = 0.42
assigned_to: str | None = None
```

- Python **ne vérifie pas** ces annotations à l'exécution
- Les éditeurs (VS Code) et des outils (mypy) s'en servent pour détecter des erreurs
- Nous les utiliserons systématiquement à partir des fonctions (S5)

Conversions de types

Fonction	Exemple	Résultat
`int()`	`int("8080")`	`8080`
`float()`	`float("0.75")`	`0.75`
`str()`	`str(42)`	`"42"`
`bool()`	`bool("")` / `bool("non")`	`False` / `True`
`round()`	`round(75.976, 1)`	`76.0`

```
>>> int("8080 ")       # espaces tolérés
8080
>>> int("80.5")
ValueError: invalid literal for int() with base 10: '80.5'
```

Console : `print()`, `input()` et f-strings

```
name = input("Nom du serveur : ")      # renvoie TOUJOURS un str
cores = int(input("Nombre de cœurs : "))

print(f"{name} possède {cores} cœurs")
```

Format	Exemple	Résultat
Décimales	`f"{75.976:.1f}"`	`76.0`
Largeur / alignement	`f"[{'web':>6}]"`	`[   web]`
Milliers	`f"{1250000:,}"`	`1,250,000`
Pourcentage	`f"{0.759:.0%}"`	`76%`
Débogage	`f"{cores=}"`	`cores=8`

🧪 À vous : Lab partie 1 (30 min)

Notebook `S02_lab_enonce.ipynb`, partie 1

- Ex. 1 ⭐ : déclarer des variables typées
- Ex. 2 ⭐ : convertir des valeurs lues dans un fichier de configuration
- Ex. 3 ⭐⭐ : formater une ligne de rapport de supervision
- Ex. 4 ⭐⭐ : traiter une saisie utilisateur

Les séquences

Type	Exemple	Modifiable ?	Usage
`str`	`"srv-web-01"`	❌ non	Texte
`list`	`["srv-01", "srv-02"]`	✅ oui	Collection qui évolue
`tuple`	`("Dell", "Latitude 5440")`	❌ non	Enregistrement fixe
`range`	`range(0, 10, 2)`	❌ non	Suite d'entiers

Points communs : `len()`, indexation `[i]`, slicing `[a:b]`, `in`, `+`, `*`

```
>>> servers = ["srv-web-01", "srv-db-01"]
>>> len(servers), servers[0], servers[-1]
(2, 'srv-web-01', 'srv-db-01')
```

Indexation et slicing

```
log  =  "2026-10-19 ERROR disk full"
index :  0123456789...
```

Expression	Résultat
`log[0:10]`	`'2026-10-19'`
`log[11:16]`	`'ERROR'`
`log[:4]`	`'2026'` (depuis le début)
`log[-4:]`	`'full'` (jusqu'à la fin)
`log[::2]`	un caractère sur deux
`log[::-1]`	la chaîne inversée

`[début:fin:pas]` : **fin exclue.**

Listes : les opérations courantes

```
servers = ["srv-web-01", "srv-db-01"]
servers.append("srv-app-01")       # ajoute à la fin
servers.insert(0, "srv-lb-01")     # insère à l'index 0
servers.remove("srv-db-01")        # supprime une valeur
last = servers.pop()               # retire et renvoie le dernier
servers.sort()                     # trie sur place
print(servers, last)
```

```
['srv-lb-01', 'srv-web-01'] srv-app-01
```

Chaînes : `split()`, `strip()`, `upper()`, `replace()`, `startswith()`, `"; ".join(liste)`

Dictionnaires et ensembles

```python
laptop = {"asset_tag": "LT-001", "brand": "Dell", "price": 3200.0}
laptop["assigned_to"] = "Amira"          # ajout ou modification
print(laptop["brand"], laptop.get("ram_gb", 16))
print(list(laptop.keys()))
```

```
Dell 16
['asset_tag', 'brand', 'price', 'assigned_to']
```

```python
team_a = {"python", "git", "docker"}
team_b = {"python", "excel"}
print(team_a & team_b, team_a | team_b, team_a - team_b)
```

Affectation par référence

Une variable est une **étiquette** posée sur un objet.

```
servers = ["srv-01", "srv-02"]
backup = servers            # même liste, deux étiquettes
backup.append("srv-03")
print(servers)
print(id(servers) == id(backup), servers is backup)
```

```
['srv-01', 'srv-02', 'srv-03']
True True
```

⚠️ Avec un objet **modifiable** (list, dict, set), modifier via une étiquette modifie pour toutes.

Copie superficielle et copie profonde

```python
import copy   # module de la bibliothèque standard (imports détaillés en S14)

site = {"name": "Tunis", "servers": ["srv-01", "srv-02"]}
shallow = site.copy()            # ou dict(site), liste.copy(), liste[:]
deep = copy.deepcopy(site)

site["servers"].append("srv-03")
print(shallow["servers"])        # partagé !
print(deep["servers"])           # indépendant
```

```
['srv-01', 'srv-02', 'srv-03']
['srv-01', 'srv-02']
```

La décomposition (unpacking)

```
hostname, ip, port = "srv-db-01;10.0.0.12;5432".split(";")
port = int(port)

brand, model = ("Dell", "Latitude 5440")

first, *others = ["srv-lb-01", "srv-web-01", "srv-web-02"]
print(first, others)

a, b = 1, 2
a, b = b, a            # échange sans variable temporaire
```

```
srv-lb-01 ['srv-web-01', 'srv-web-02']
```

Pièges fréquents

Piège	Exemple	Correction
`input()` renvoie du texte	`input() + 1` → TypeError	`int(input()) + 1`
Tuple à un élément	`("srv-01")` est un str	`("srv-01",)`
Méthode de liste et retour	`l = l.sort()` → `None`	`l.sort()` ou `sorted(l)`
Clé absente	`d["ram"]` → KeyError	`d.get("ram", 0)`
Alias involontaire	`b = a` puis `b.append(...)`	`b = a.copy()`
`{}` vide	c'est un dict	`set()` pour un ensemble vide

🧪 À vous : Lab partie 2 (30 min)

Notebook `S02_lab_enonce.ipynb`, parties 2 et 3

- Ex. 5 ⭐ : découper une ligne de log par slicing
- Ex. 6 ⭐⭐ : gérer une liste de serveurs
- Ex. 7 ⭐⭐ : dictionnaire d'équipement et ensembles de compétences
- Ex. 8 ⭐⭐⭐ : références et copies
- Ex. 9 ⭐⭐ : unpacking
- Synthèse : analyser une ligne d'export de tâches projet
- **GestParc – étape 1** : représenter l'inventaire avec des dictionnaires

Récapitulatif

- Types de base : `int`, `float`, `str`, `bool`, `None` ; typage **dynamique et fort**
- `type()`, `isinstance()`, conversions explicites `int()`, `float()`, `str()`
- f-strings : `f"{valeur:format}"`
- Séquences : `str`, `list`, `tuple`, `range` + slicing `[début:fin:pas]`
- `dict` (clé → valeur) et `set` (éléments uniques)
- Variables = étiquettes : attention aux objets modifiables, `copy()` / `deepcopy()`
- Unpacking : `a, b = b, a`, `first, *others = ...`

Séance 3 : conditions et boucles, menu interactif de GestParc